学 Java Concurrency 时，Lock 相关的名字很容易让大家搞混：Mutex、Read-Write Lock、Spin Lock、Pessimistic Lock、Optimistic Lock、CAS、AQS、`synchronized`、`ReentrantLock`、`StampedLock`、Biased Lock、Lightweight Lock、Heavyweight Lock。

这些名字并不都在同一个分类维度里。

有的说“谁能进入 Critical Section”，比如 Mutex 和 Read-Write Lock；有的说“拿不到 Lock 时怎么等”，比如 Spin Lock 和 Blocking Lock；有的说“修改 Shared 数据前先 Lock 住，还是提交时再校验”，比如 Pessimistic Lock 和 Optimistic Lock；还有的说 HotSpot 在不同竞争强度下怎么优化 `synchronized`。

这篇文章先把 Lock 的坐标系立起来，再看 Java 里常用 Lock 工具怎么落地。关于 Pessimistic Lock、Optimistic Lock 和 CAS 的细节，站内已经有两篇文章详细介绍：Optimistic Lock 和 Pessimistic Lock 详解、CAS 详解。本文只保留必要 Context，重点放在“Lock 体系怎么串起来”。

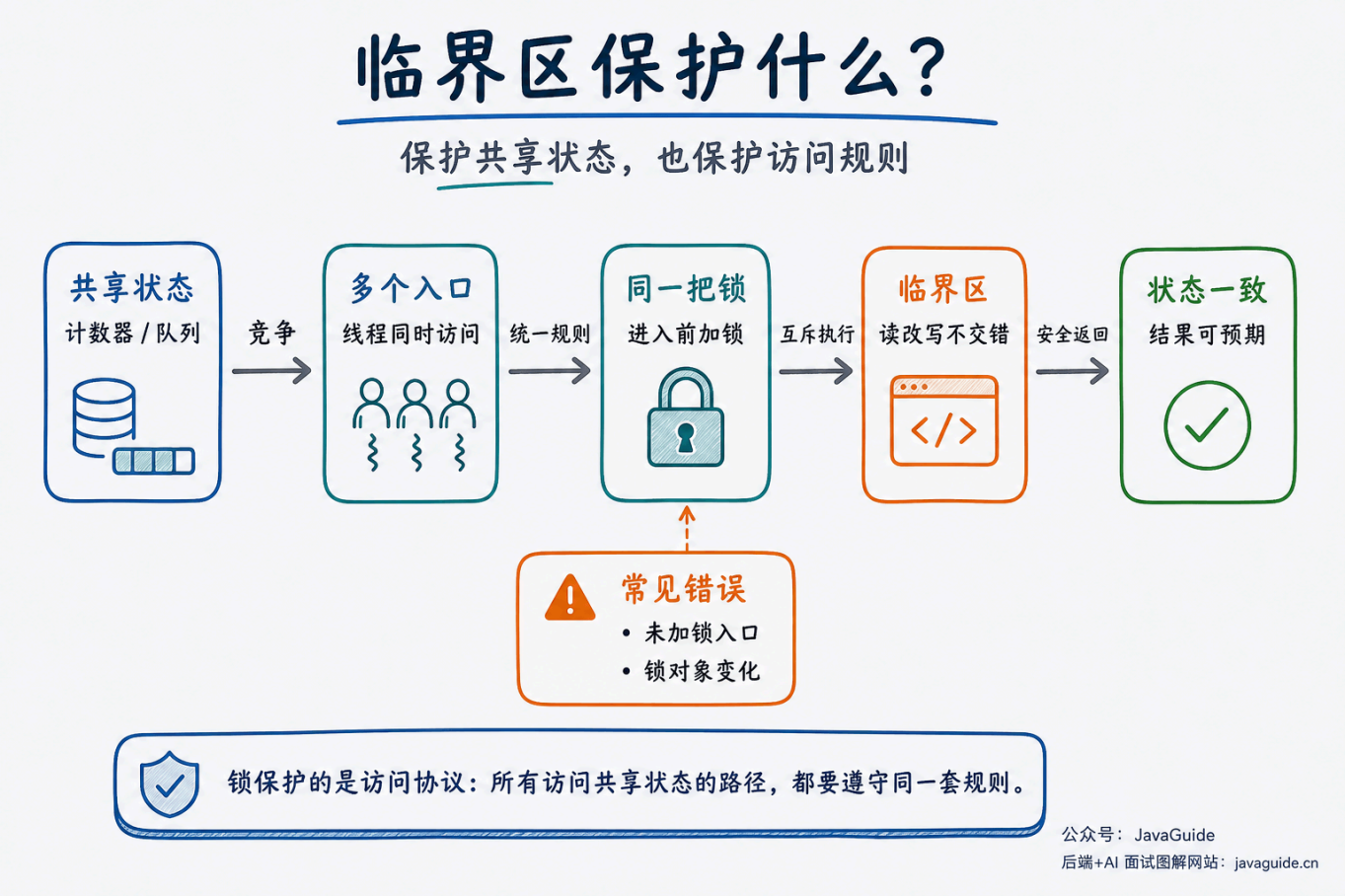
PS：本文主要以 HotSpot / OpenJDK 为背景。`synchronized` 的 monitor Mutual Exclusion 和内存语义属于 Java/Java Virtual Machine (JVM) 规范层面；Object Header、Mark Word、Lightweight Lock、Lock 膨胀这些内容属于 HotSpot 实现优化，Java 语言规范并不承诺固定流程。 Biased Lock 从 JDK 15 起默认禁用并废弃相关 Parameter，JDK 18 起相关 Parameter 已经 obsoleted；Virtual Thread 与 `synchronized` pinning 的结论也要区分 JDK 21~23 和 JDK 24+。

先用一张表把分类维度拆开：

维度	典型名称	回答的问题
Critical Section Mutual Exclusion 方式	Mutex、Read-Write Lock	谁能进入 Critical Section
等待策略	Spin Lock、Blocking Lock	拿不到 Lock 时怎么等
Concurrency 控制思路	Pessimistic Lock、Optimistic Lock	先 Lock 住再改，还是提交时校验
Atomic Update 机制	CAS、Atomic Class	如何无 Blocking 更新单个 Variable
JVM 实现优化	Lightweight Lock、Heavyweight Lock、Lock 膨胀	HotSpot 如何降低 `synchronized` 成本
Java Lock 工具	`synchronized`、`ReentrantLock`、`StampedLock`	代码里具体用什么

一把 Lock 到底保护什么？

Lock 要解决的是 Critical Section 问题。 Critical Section 指那段会访问 Shared 可变状态，并且不能让多个执行单元随意交错执行的代码。



比如下面这个自增：

```
count++;
```

源码里只有一行，但这行代码不能当成不可拆的动作。 Thread 通常要先读出 `count`，再加 1，最后写回去。两个 Thread 同时执行时，可能都读到旧值 `0`，各自算出 `1`，最后都把 `1` 写回去。两个 Thread 都执行了自增，结果只加了一次。

Lock 的做法很直接：进入这段代码前先获得 Lock，执行完再 Release Lock。只要所有访问同一份 Shared 状态的代码都遵守同一把 Lock 的约定，就可以把原本可能交错的读写压成一段 Mutual Exclusion 执行。

这里有一句很容易被忽略的话：**Lock 真正保护的是访问 Object 状态的协议，Object 本身不会因为被 Locking 就自动安全**。

`synchronized (account)` 不会神奇地让 `account` 的所有 Field 都安全。如果另一段代码绕过这把 Lock 直接改 `account.balance`，Thread-safe 照样会被破坏。MIT 6.005 的 Lock 课程反复强调的也是这个点：Lock 应该守住某个数据抽象的表示不 Variable，随手找个 Object 套一下，并不能保证不 Variable 一直成立。

Mutex：同一时刻只允许一个 Thread 进入

Mutex 的规则很简单：同一时刻，最多只有一个 Thread Hold Lock 并进入 Critical Section。

在 Java 中，`synchronized` 和 `ReentrantLock` 都可以作为 Mutex 使用：

```
class Counter {
    private int count;

    public synchronized void increment() {
        count++;
    }

    public synchronized int get() {
        return count;
    }
}
```

换成 ReentrantLock，写法会啰嗦一点，但能拿到更多控制权：

```
class Counter {
    private final ReentrantLock lock = new ReentrantLock();
    private int count;

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock();
        }
    }
}
```

try/finally 不能省。synchronized 的释放动作由 JVM 帮你做，Code Block 正常退出或 Exception 退出都会释放 monitor；ReentrantLock 是显式 API，拿 Lock 和 Unlock 要自己配对。Oracle 的 ReentrantLock 文档也把“调用 lock 后立刻进入 try 块”作为推荐写法。

Mutex 真正难的地方不在语法，而在 Lock 粒度。

一把大 Lock 把所有操作都包住，最省心，但 Concurrency 度低；多把小 Lock 分别保护不同数据，吞吐可能更好，但 Lock 顺序、Deadlock、状态一致性都更难管。OSTEP 在讲 POSIX mutex 时也提到过这个取舍：不同数据用不同 Lock 能增加 Concurrency，但程序员必须清楚每把 Lock 到底保护哪一块状态。

Read-Write Lock：读读 Shared，Write Operation Exclusive

Mutex 对 Read Operation 也很严格：只要一个 Thread 在读，另一个 Thread 也不能进来读。但很多业务 Object 有一个特点：读不会改变状态，多个读 Thread 同时执行并不会互相破坏。

Read-Write Lock 就是为这种场景准备的。

它把访问分成两 Class：

- Read Lock：Shared Lock，多个 Thread 可以同时持有。

- Write Lock：Exclusive Lock，只能一个 Thread 持有，并且 Write Lock 和 Read Lock Mutual Exclusion。

对应规则也很好记：

- 读读 Non-exclusive。
- 读写 Mutual Exclusion。
- 写写 Mutual Exclusion。

Java 里的典型实现是 `ReentrantReadWriteLock`：

```
class ProfileCache {
    private final ReentrantReadWriteLock rwLock = new
ReentrantReadWriteLock();
    private final Lock readLock = rwLock.readLock();
    private final Lock writeLock = rwLock.writeLock();
    private final Map<Long, String> cache = new HashMap<>();

    public String get(long userId) {
        readLock.lock();
        try {
            return cache.get(userId);
        } finally {
            readLock.unlock();
        }
    }

    public void put(long userId, String profile) {
        writeLock.lock();
        try {
            cache.put(userId, profile);
        } finally {
            writeLock.unlock();
        }
    }
}
```

Read-Write Lock 适合读多写少、Read Operation 足够短、Data Structure 不容易被拆坏的场景。它不适合所有缓存，也不一定比 Mutex 快。如果 Write Operation 很频繁，读 Thread 和写 Thread 会不断互相挡路，Read-Write Lock 的维护成本反而可能抵消收益。

Java 还提供了 `StampedLock`，它支持 Write Lock、Pessimistic Read Lock 和 Optimistic Read。 Optimistic Read 没有真正持有传统 Read Lock；它会先拿一个 stamp，读完后再校验期间有没有写入发生：

```
class Point {
    private final StampedLock lock = new StampedLock();
    private double x;
    private double y;

    double distanceFromOrigin() {
```

```
            long stamp = lock.tryOptimisticRead();
            double currentX = x;
            double currentY = y;
            if (!lock.validate(stamp)) {
                stamp = lock.readLock();
                try {
                    currentX = x;
                    currentY = y;
                } finally {
                    lock.unlockRead(stamp);
                }
            }
            return Math.hypot(currentX, currentY);
        }
    }
```

`StampedLock` 的 Optimistic Read 有边界：读到的数据可能短暂不一致，所以只适合能在本地 Variable 里完成读取、并且可以通过 `validate` 失败后重读来兜底的短读场景。它也很难直接替代 `ReentrantReadWriteLock`，尤其要注意它不支持 Reentrant。

Spin Lock：不 Blocking，先原地等一会儿

Thread 拿不到 Lock 时，通常有两种等待方式：

- Blocking：挂起 Current Thread，让 Operating System 之后再唤醒。
- Spin：不挂起 Thread，在 CPU 上 Loop 检查 Lock 是否可用。

Spin Lock 适合 Critical Section 非常短的场景。比如持 Lock Thread 马上就会 Release Lock，如果等待 Thread 直接 Blocking，Thread 挂起和唤醒的成本可能比“原地转几圈”等待还高。

问题也在这里：Spin 会付出 CPU 成本，它会持续占用 CPU。如果 Lock 很久不释放，或者等待 Thread 很多，Spin 会把 CPU 时间浪费在空转上。

Java 代码里可以用 CAS 写出一个很小的 Spin Lock 示例：

```
class SpinLock {
    private final AtomicReference<Thread> owner = new AtomicReference<>();

    public void lock() {
        Thread current = Thread.currentThread();
        while (!owner.compareAndSet(null, current)) {
            Thread.onSpinWait();
        }
    }

    public void unlock() {
        Thread current = Thread.currentThread();
        if (!owner.compareAndSet(current, null)) {
            throw new IllegalMonitorStateException();
        }
```

```
        }
    }
```

这段代码只是用来说明“Spin + CAS”的关系，不建议直接拿去做业务 Lock。真实 Lock 要考虑 Reentrant、Fairness、Interrupt、Timeout、Exception-safe Release、Monitoring Metric、Wait Queue 等问题。JDK 已经把这些复杂性 Encapsulate 在 `synchronized`、`ReentrantLock`、AQS Synchronizer 和 Atomic Class 里了。

Pessimistic Lock、Optimistic Lock 和 CAS 的位置

Pessimistic Lock 和 Optimistic Lock 描述的是两种 Concurrency 控制思路，不对应某个固定 Java Class。

Pessimistic Lock 假设冲突很可能发生，所以先把资源 Lock 住再操作。`synchronized`、`ReentrantLock`、Database `SELECT ... FOR UPDATE` 都是常见例子。

Optimistic Lock 假设冲突不频繁，先不 Blocking 别人，提交修改时再检查数据有没有被改过。 Database 里的 `version` Field、Java Atomic Class 里的 CAS，都属于这个方向。

CAS（Compare-And-Swap，CAS）可以理解成一种硬件支持的 Atomic Update 方式：只有当内存中的值仍然等于预期旧值时，才把它改成新值。否则说明有人先改过了，Current Thread 可以选择 Retry、放弃或走 Fallback 逻辑。

CAS 常见问题有三个：

- 失败 Retry 会消耗 CPU，冲突越激烈越明显。
- 只能很自然地处理单个 Variable，多个 Variable 的一致性要额外设计。
- ABA 问题：值从 A 变成 B，又变回 A，单看值会以为它没变过。

ABA 可以用 Version、Timestamp 或带标记 Reference 解决。Java 里有 `AtomicStampedReference` 和 `AtomicMarkableReference` 这类工具，不过在业务代码里更常见的做法是让数据模型本身带 Version。

这块如果继续展开，就会和已有文章重复。想看实现方式、Version 示例、ABA 处理和 Atomic Class 源码，可以继续读：

- Optimistic Lock 和 Pessimistic Lock 详解
- CAS 详解
- Atomic Class 总结

synchronized 的底层：monitor、Bytecode 和内存语义

`synchronized` 是 Java 语言内置的 Synchronization Mechanism，可以修饰 Instance Method、Static Method，也可以包住 Code Block。

```
class Account {
    private long balance;

    public synchronized void deposit(long amount) {
        balance += amount;
    }
```

```
    public long balance() {
        synchronized (this) {
            return balance;
        }
    }
}
```

Synchronized Method 和 Synchronized Block 在 Bytecode 层面的表现不完全一样：

- Synchronized Method 依赖 Method Access Flag `ACC_SYNCHRONIZED`。
- Synchronized Block 会生成 `monitorenter` 和 `monitorexit` Instruction。

不管表现形式如何，语义都是进入 monitor、退出 monitor。Java 语言规范还规定了 Lock 释放和后续 Lock 获取之间的 happens-before 关系：一个 Thread 释放某把 Lock 之前的写入，对随后获得同一把 Lock 的 Thread 可见。

这也是 `synchronized` 和"只做 Mutual Exclusion"的普通概念 Lock 不同的地方。它同时提供 Mutual Exclusion 和 Memory Visibility。只保护 Critical Section 但不建立 Visibility，另一个 Thread 可能仍然读到旧值。

另外，`synchronized` 是 Reentrant 的。一个 Thread 已经持有某个 Object 的 monitor 时，可以再次进入同一把 Lock 保护的代码，JVM 会记录 Reentrancy Count，退出时逐层减少。

```
class ReentrantDemo {
    public synchronized void outer() {
        inner();
    }

    public synchronized void inner() {
        // 同一线程可以再次进入 this 的 monitor
    }
}
```

synchronized Lock 优化：别把"Lock Inflation"背成固定口诀

很多资料会把 `synchronized` 讲成"Lock-Free -> Biased Lock -> Lightweight Lock -> Heavyweight Lock"。这条线索对理解 HotSpot 早期优化很有帮助，但不能脱离版本。

JDK 6 之后，HotSpot 为 `synchronized` 做了大量优化。 Biased Lock 面向"总是同一个 Thread 进入同一把 Lock"的场景；Lightweight Lock 面向"竞争不激烈，多个 Thread 错开进入"的场景；Heavyweight Lock 则会用到 ObjectMonitor，竞争 Thread 可能 Blocking 和唤醒。

版本差异要单独记一下：

- JDK 6 到 JDK 14：Biased Lock 是 HotSpot 常见优化之一。
- JDK 15：JEP 374 将 Biased Lock 默认禁用，并废弃相关 Parameter。
- JDK 18：Biased Lock 相关 Parameter 被 obsoleted，传入后会被忽略并给出警告。
- JDK 21 到 JDK 23：Virtual Thread 在 `synchronized` 中 Blocking 时可能 pin 住 Platform Thread。

- JDK 24：JEP 491 改进了 Virtual Thread 与 `synchronized` 的配合，Blocking 在 `synchronized` 上的 Virtual Thread 可以释放底层 Platform Thread，减少 pinning 问题。

所以，面试或写文章时可以讲“HotSpot 曾经通过 Biased Lock、Lightweight Lock、Heavyweight Lock 降低 `synchronized` 成本”，但不要把 Biased Lock 说成现代 JDK 一定会走的默认路径。

工程上更重要的是另一个结论：早年那句“能不用就不用”已经不适合今天的 `synchronized`。普通 Mutual Exclusion 场景下，它语法简单、Exception-safe Release 安全、Just-In-Time Compilation (JIT) 优化成熟。只有当你需要 Fair Lock、Interruptible 获取、Timeout 获取、多个 Condition Queue 时，才更自然地转向 `ReentrantLock`。

ReentrantLock、Condition 和 AQS 怎么接上

`ReentrantLock` 提供了比 `synchronized` 更细的控制能力：

- 可以选择 Fair Lock 或 Nonfair Lock。
- 可以用 `lockInterruptibly()` 响应 Interrupt。
- 可以用 `tryLock()` 或 `tryLock(timeout, unit)` 避免无限等待。
- 可以创建多个 `Condition`，把不同等待条件拆开管理。

一个典型写法如下：

```
class BoundedBuffer<E> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();
    private final Queue<E> queue = new ArrayDeque<>();
    private final int capacity;

    BoundedBuffer(int capacity) {
        this.capacity = capacity;
    }

    public void put(E item) throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (queue.size() == capacity) {
                notFull.await();
            }
            queue.add(item);
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    public E take() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (queue.isEmpty()) {
                notEmpty.await();
            }
```

```
                E item = queue.remove();
                notFull.signal();
                return item;
            } finally {
                lock.unlock();
            }
        }
    }
```

这里的 `while` 也不能随便改成 `if`。 Thread 被唤醒后，只能说明“有机会重新竞争 Lock 并检查条件”，不代表条件一定成立。虚假唤醒、多个等待 Thread 竞争、条件被别的 Thread 先消费掉，都要求醒来后再次检查。

`ReentrantLock` 底层依赖 AQS（AbstractQueuedSynchronizer）。AQS 可以先粗略理解成一套 Synchronizer 框架：用一个 `state` 表示 Synchronization State，用 FIFO Queue 管理没抢到资源的 Thread，再配合 CAS、`LockSupport.park/unpark` 完成排队、Blocking 和唤醒。

很多 Concurrency 工具都建立在 AQS 之上，比如 `ReentrantLock`、`Semaphore`、`CountDownLatch`、`ReentrantReadWriteLock`。如果想把 Queue、`state`、CAS 和 Blocking 唤醒这条线继续拆开，可以接着看 AQS 详解 和 从ReentrantLock的实现看AQS的原理及应用。

Java Lock 该怎么选？

选 Lock 时别急着比较“哪个最快”。 Lock 的表现跟 Critical Section Length、竞争强度、Thread Count、失败后的处理方式都有关。先把几个问题问清楚：这段代码保护哪份 Shared 状态？持 Lock 时间大概多长？拿不到 Lock 时能不能等待？等待失败后是返回、Retry，还是直接报错？

如果只是保护 JVM Process 内的一小段状态，比如更新几个 Field、维护一个内存 Map、切换 Object 状态，`synchronized` 往往就够了。它写起来短，退出 Code Block 时自动 Release Lock，也少了手写 `unlock()` 漏掉的风险。等到代码需要 Timeout 获取、Interruptible 获取、Fair Lock，或者要用多个 `Condition` 管理不同 Wait Queue，再换成 `ReentrantLock` 会更顺手。

读多写少时，可以看 `ReentrantReadWriteLock`。这里的重点是“写少”，光有读 Method 还不够。如果 Write Operation 很频繁，Read Lock 和 Write Lock 会一直互相挡，维护读写状态也有成本，最后未必比一把 Mutex 更划算。`StampedLock` 的 Optimistic Read 更挑场景：读逻辑要短，读到中间状态也不能出大问题，并且必须接受校验失败后重新读一遍。

如果只是更新一个计数、状态位或 Reference，优先看 Atomic Class、`LongAdder`、`LongAccumulator` 这类工具。它们适合很短的 Atomic Update，不适合把一整段业务流程塞进 CAS Retry Loop 里。业务流程越长，失败 Retry 越容易把 CPU 消耗在无效 Loop 上，也更难处理副作用。

如果问题已经越过 JVM 边界，比如多个应用 Instance 同时改同一行 Database 记录，Java 里的 Lock 就管不住了。冲突不频繁时，可以用 Version 做 Optimistic Lock；冲突比较频繁、必须强一致修改时，通常要回到 Database Row Lock、`SELECT ... FOR UPDATE`、Unique Constraint 这类 Database 机制。再往外走到跨服务 Mutual Exclusion，就需要 Redis、ZooKeeper、Database 等外部系统来承接，不能指望 `synchronized` 或 `ReentrantLock`。

Lock 粒度也别一味追求“小”。一把大 Lock 容易保证正确性，但吞吐可能受影响；拆成多把小 Lock，竞争会少一些，可 Lock 顺序、Deadlock 和排查成本都会上来。很多时候，先用清楚的一把 Lock 把不 Variable 守住，再根据压测结果拆 Lock，比一开始就设计一 Heap 细粒度 Lock 更稳。

常见坑

Lock Object 不稳定。

有些代码看起来加了 Lock，实际可能 Lock 到了不同 Object。常见原因是 Lock Object 会变，比如 String 拼接结果、装箱 Object、可重新赋值的 Field。 Thread A 进来时 Lock 的是旧 Object，Thread B 进来时 Lock 的是新 Object，两边互不影响，Critical Section 就被拆开了。

```
private Object lock = new Object();

public void update() {
    synchronized (lock) {
        lock = new Object(); // 后续线程可能会锁到另一把锁
    }
}
```

如果需要单独的 Lock Object，通常把它定义成 `private final`，并且不要把它暴露给外部代码。

Lock 住外部可见 Object。

`synchronized (this)` 和 `synchronized (SomeClass.class)` 有时没问题，但它们也可能被外部代码拿来 Locking，导致你无法控制 Lock Contention 范围。库代码尤其要谨慎，通常更推荐私有 final Lock Object。

```
private final Object lock = new Object();
```

持 Lock 期间做慢操作。

持 Lock 时访问 Database、调用 RPC、写大文件，都会拉长 Lock 占用时间。 Lock 占用越久，等待 Thread 越多，Timeout、Thread Pool 耗尽和 Deadlock 风险都会变高。

Lock 顺序不一致。

两个 Thread 分别按 `A -> B` 和 `B -> A` 的顺序 Locking，很容易形成 Deadlock。多把 Lock 同时使用时，要给资源排一个全局稳定顺序。 Deadlock 的完整介绍可以看 Deadlock 详解。

把 Thread-safe Class 和 Compound Operation 混为一谈。

`ConcurrentHashMap` 的单次 `get`、`put` 是 Thread-safe 的，但“先判断不存在，再插入”是 Compound Operation，需要用 `computeIfAbsent` 这类 Atomic Method，或者额外同步。

```
// 不推荐：containsKey 和 put 之间可能被其他线程插入
if (!map.containsKey(key)) {
    map.put(key, createValue());
}
```

```
// 推荐：把复合逻辑交给 ConcurrentHashMap 的原子方法
map.computeIfAbsent(key, ignored -> createValue());
```

只看 Lock，不看资源池。

线上很多“卡住”和 Java Lock Deadlock 无关，Thread 可能都在等 Database Connection、HTTP 连接、Thread Pool Queue 或外部服务返回。 Thread Stack 里看到 `WAITING` 只能说明 Thread 在等，判断 Deadlock 还要找到稳定的等待环。

总结

Lock 是一组 Concurrency 控制工具的总称，这个概念还是比较大的。

Mutex 和 Read-Write Lock 回答“谁能进 Critical Section”；Spin Lock 和 Blocking Lock 回答“拿不到 Lock 怎么等”；Pessimistic Lock 和 Optimistic Lock 回答“冲突发生前后怎么处理”；CAS 和 Atomic Class 解决单 Variable Atomic Update；`synchronized`、`ReentrantLock`、`StampedLock`、AQS 则是 Java 把这些思想落到代码里的方式。

真正写代码时，先把 Shared 状态和不 Variable 找出来，再决定 Lock 保护什么、Lock 粒度多大、等待是否能 Interrupt、失败是否能 Retry。工具只是手段，真正要守住的是同一套同步协议：所有访问 Shared 状态的路径都必须遵守它。

参考资料

- The Java Language Specification, Chapter 17. Threads and Locks
- The Java Virtual Machine Specification, monitorenter
- Oracle Java API: Lock
- Oracle Java API: ReentrantLock
- Oracle Java API: StampedLock
- OpenJDK JEP 374: Deprecate and Disable Biased Locking
- OpenJDK JEP 491: Synchronize Virtual Threads without Pinning
- OSTEP 中文版：Locks
- MIT 6.005: Locks and Synchronization